



Memory and Storage Needs of LLMs

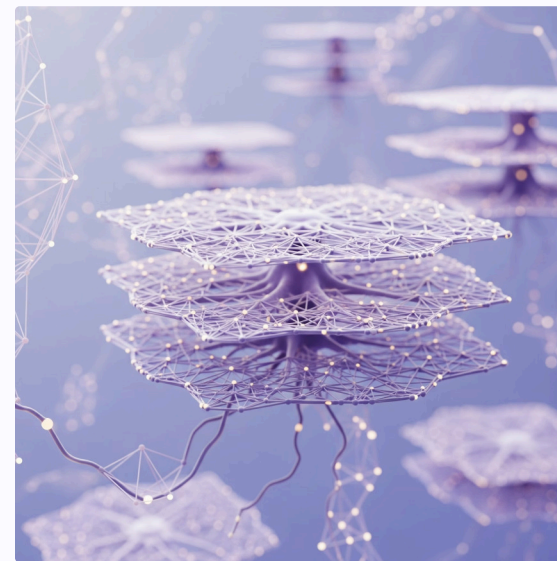
Managing Parameters, Activations, and Checkpoints at Scale

An examination of the critical infrastructure requirements for training and deploying billion to trillion parameter language models.

Why Memory & Storage Are Critical

Modern LLMs face unprecedented resource demands:

- **Billions–trillions of parameters** requiring enormous memory capacity
- Training stores parameters + optimizer states + activations simultaneously
- Inference requires fast weight loading and context caching capabilities
- Checkpoint storage costs grow exponentially with model size and training runs



Memory Demands in Training

Parameter Storage

1 billion parameters $\approx$ 4 GB in FP32 precision

Optimizer states (Adam) add 4 $\times$ overhead:
 $\sim$ 16 GB per billion parameters

Activation Memory

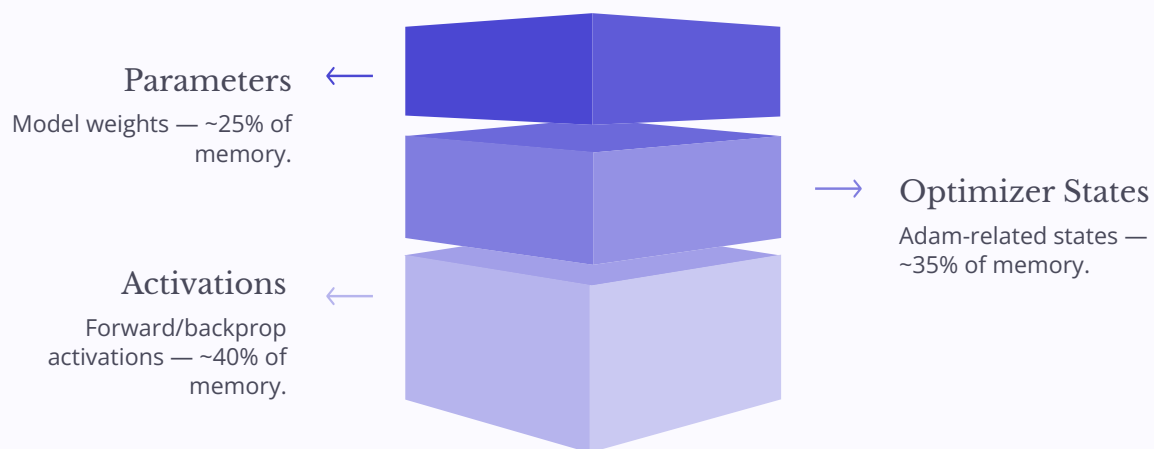
Forward pass activations often exceed parameter size during backpropagation

Gradient checkpointing trades compute for memory by recomputing activations

Precision Tradeoffs

Mixed precision training (FP16/BF16) cuts memory footprint by approximately 50%

Maintains numerical stability while reducing VRAM requirements



Storage Demands in Training

Dataset Storage

Petabyte-scale datasets
require specialized
distributed file systems

High throughput needed to
avoid I/O bottlenecks during
training

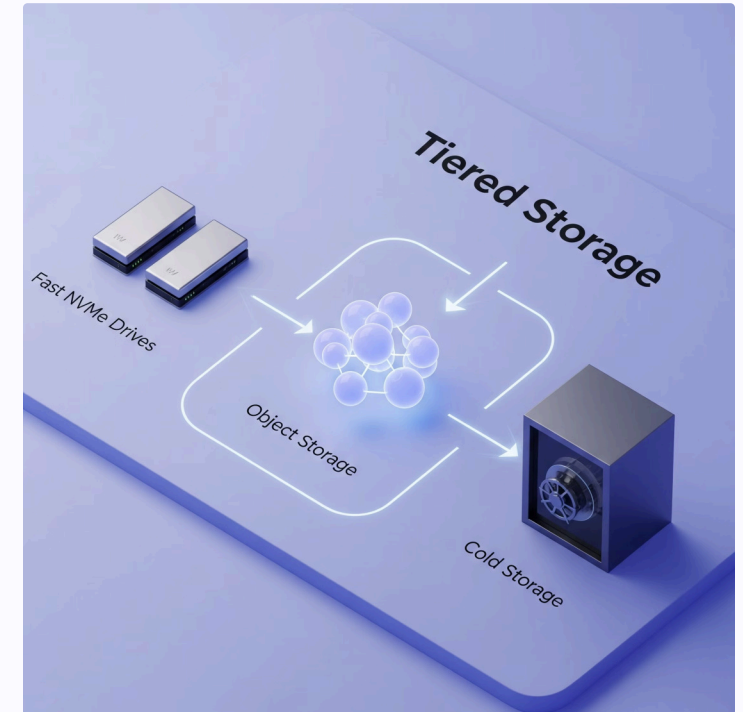
Tiered Architecture

Requires NVMe for hot data + object store for checkpoints + cold storage for archives

Checkpoint Volume

Frequent checkpoints
generate **1–10 TB per
training run**

Critical for resuming training
and experiment tracking



Inference-Time Memory Needs

Weight Loading

Serving requires loading large model weights quickly from storage to GPU memory

Loading time directly impacts initialization latency

KV-Cache Growth

Key-value cache for attention **grows linearly with sequence length**

Can exceed parameter size for long contexts (>4K tokens)

Optimization Techniques

Batch inference amortizes memory usage across multiple requests

Quantization (INT8/4-bit) reduces serving footprint by 2-8×



Storage for Model Lifecycle

Training

Initial model development with full-precision weights and optimizer states

Archiving

Cold storage with retention policies to manage costs



Fine-tuning

Domain adaptation requiring multiple checkpoint variants

Deployment

Optimized serving formats with versioning for reproducibility

Model registries like **MLflow and Weights & Biases** track artifacts throughout this lifecycle, enabling systematic version control and efficient retrieval.

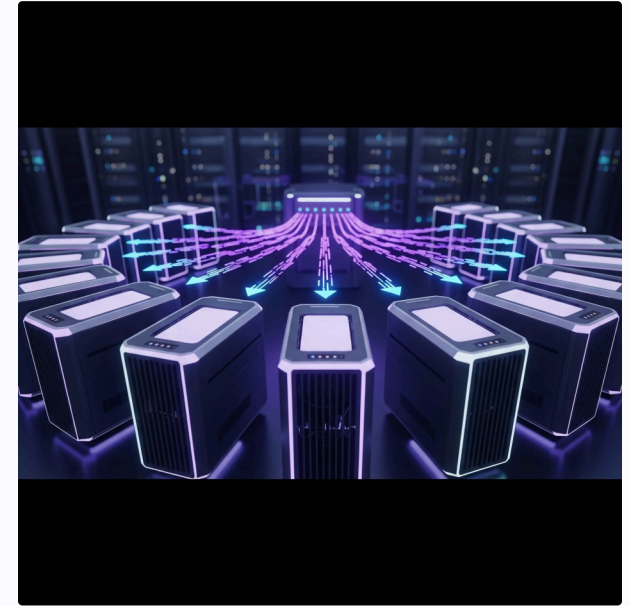
Optimization Strategies

Distributed Training Optimizations

- **Parameter Sharding** (ZeRO, FSDP, Megatron-LM) distributes model across GPUs
- **Activation Offloading** moves tensors to CPU/NVMe when not needed
- **Gradient Accumulation** reduces optimizer state memory pressure

Storage Optimizations

- **Compression & Deduplication** reduces dataset footprint
- **Differential Checkpoints** store only parameter changes
- **Checkpoint Pruning** removes intermediate saves systematically



Memory and storage optimizations work together to enable training models that would otherwise be impossible with current hardware.

Key Takeaways



Memory: Real-time Bottleneck

During training, parameters + optimizer states + activations must fit in available memory

During inference, weight loading speed and KV-cache growth determine throughput



Storage: Long-term Burden

Checkpoints and datasets quickly reach petabyte scale for large models

Versioning and reproducibility requirements compound storage needs



Implementation Strategy

Build tiered infrastructure that balances performance, cost, and accessibility

Implement systematic lifecycle management with clear retention policies

Leverage distributed training techniques to scale beyond single-node limitations